

Arquitectura de HiveAI - User story: generar y correr una ejecución

PC del usuario - App de escritorio (Deno Desktop)

Frontend (React + Vite)

UI / vista

ExecutionsMain.tsx, ExecutionChatPanel.tsx,
RunExecutionModal.tsx, visual-builder/index.tsx

Estado de vista

useExecutionWorkspace.ts, useGraphEditor.ts

Cliente API

generateExecution.ts, runExecutionStream.ts

HTTP + SSE

1 POST /api/executions/generate y /:id/run

eventos SSE en vivo:

planning, node_added, done 8

Backend (Deno + Hono, localhost:8000)

Controller

executions/router.ts

Casos de uso

generate-execution.ts, run-execution.ts,
update-execution-graph.ts

Dominio (Visual Builder)

generate-graph.ts, validate-graph.ts, compiler.ts,
llm-executor.ts, plugin-node-executor.ts

Microkernel de plugins

hive-microkernel.ts

run-shell

web-search

file-read

file-search

web-read

file-ops

plugins externos (proceso aislado)

Persistencia

execution-repository.ts



executions
execution_graphs
execution_history

- Componente propio
- Componente de terceros
- Base de datos local
- Llamada directa
- > Evento SSE / consulta externa

compila y ejecuta el grafo

@langchain/langgraph
(StateGraph)

Ollama

proceso local, localhost:11434

Servidor HTTP

POST /api/chat

delega inferencia

Runner llama.cpp

inferencia CPU/GPU

carga pesos

Modelo de IA local

pesos GGUF en disco

4 prompts del generador
y nodos LLM